

Python Language Lab

Complete Viva Voice Q&A Guide

Covers Syllabus Topics 1-10 + Teacher Favourite Questions

Section	What is Covered
SECTION A	Syllabus-Based Q&A — All 10 Lab Topics with Code
SECTION B	Additional Viva Questions Teachers Commonly Ask
BONUS	Quick Cheat Sheet for Last-Minute Revision

Exam Tips: Always explain your code line-by-line. Mention time complexity for sorting. Know the difference between similar concepts. Be confident — the examiner wants to see your understanding, not just memorization.

SECTION A — Syllabus-Based Viva Questions (Topics 1 to 10)

Topic 1 — Basic Data Types in Python

Q: What are the basic built-in data types in Python?

A: Python has: **int** (whole numbers), **float** (decimals), **str** (text), **bool** (True/False), **complex** (e.g. 3+4j), **list**, **tuple**, **set**, **dict**, and **NoneType**.

Q: What is the difference between int and float?

A: int stores whole numbers like 5, -3. float stores decimal numbers like 3.14, -0.5. Floats occupy more memory. Division of two ints in Python 3 gives a float.

Q: How do you check the data type of a variable?

A: Using the built-in **type()** function. For stricter type checking, use **isinstance()**.

```
x = 42
print(type(x))           #
print(isinstance(x, int)) # True
```

Q: What is type casting? Give examples.

A: Converting one type to another using **int()**, **float()**, **str()**, **bool()**. Example: **int('5')** gives 5. **str(3.14)** gives '3.14'. **int(True)** gives 1.

Q: Is Python statically typed or dynamically typed?

A: Dynamically typed — you do NOT declare the type of a variable. The type is determined automatically at runtime based on the assigned value.

Q: What is the NoneType?

A: None is Python's null value, representing the absence of a value. **type(None)** returns **NoneType**. Functions that don't return anything return None by default.

Topic 2 — Distance Between Two Points & Command-Line Arguments

Q: What is the formula to find distance between two points?

A: Euclidean distance = $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$. In Python use **math.sqrt()** or raise to power 0.5.

```
import math
x1,y1 = float(input()), float(input())
x2,y2 = float(input()), float(input())
d = math.sqrt((x2-x1)**2 + (y2-y1)**2)
print('Distance:', d)
```

Q: How do you accept command-line arguments in Python?

A: Using **sys.argv** from the **sys** module. **sys.argv[0]** is the script name; **sys.argv[1]**, **sys.argv[2]**, ... are the arguments passed by the user.

```
# add.py
import sys
a = int(sys.argv[1])
b = int(sys.argv[2])
print('Sum =', a + b)
# Run: python add.py 5 10
```

Q: What happens if sys.argv arguments are missing?

A: Python raises an **IndexError** because the list index is out of range. Handle it with **try-except** or check **len(sys.argv)** before accessing.

Q: What is the difference between input() and sys.argv?

A: input() reads from the keyboard during program execution (interactive). sys.argv reads arguments passed when launching the script from the terminal (non-interactive, great for automation).

Topic 3 — Even/Odd Check & For Loop with Decimal Fractions

Q: How do you check if a number is even or odd?

A: Using the modulo operator %. If $n \% 2 == 0$ it is even, otherwise odd.

```
n = int(input('Enter number: '))
if n % 2 == 0:
    print(n, 'is Even')
else:
    print(n, 'is Odd')
```

Q: What is the modulo (%) operator?

A: % returns the remainder after division. E.g. $10 \% 3 = 1$. Used to check divisibility, cycle through values, and more.

Q: Write the program to print 1/2, 1/3, ..., 1/10 as decimals using a for loop.

```
for i in range(2, 11):
    print(f'1/{i} = {1/i:.4f}')
```

Q: How does range() work? Explain range(2, 11).

A: range(start, stop) generates integers from start up to — but NOT including — stop. range(2,11) gives 2,3,4,5,6,7,8,9,10. range() also accepts a third step argument.

Q: What is an f-string?

A: A formatted string literal (Python 3.6+). Prefix the string with f and put expressions inside {}. .4f means format as float with 4 decimal places.

Topic 4 — List, Tuple, For Loop, While Loop

Q: What is the key difference between a list and a tuple?

A: **List**: mutable (can add, remove, change elements), defined with []. **Tuple**: immutable (cannot change after creation), defined with (). Tuples are faster, use less memory, and can be used as dictionary keys.

```
my_list = [1, 2, 3] # mutable
my_tuple = (1, 2, 3) # immutable
my_list[0] = 99 # OK
# my_tuple[0] = 99 # TypeError!
```

Q: How do you loop over a sequence using a for loop?

A: The for loop iterates over any iterable — list, tuple, string, dict, range, etc.

```
fruits = ['apple', 'banana', 'mango']
for fruit in fruits:
    print(fruit)
```

Q: What is the difference between for loop and while loop?

A: **for loop**: used when the number of iterations is known or you iterate over a sequence. **while loop**: used when iterations depend on a condition being True — number of iterations may not be known in advance.

Q: Write a while loop that counts down from a user-entered number to 0.

```
n = int(input('Enter number: '))
while n >= 0:
```

```
print(n)
n -= 1
print('Done!')
```

Q: What happens if you forget to update the variable in a while loop?

A: You get an **infinite loop** — the program never terminates. Always ensure the loop condition eventually becomes False.

Topic 5 — Sum of Primes & Even Fibonacci Terms

Q: What is a prime number? How do you check if a number is prime?

A: A prime has exactly two divisors: 1 and itself. To check, divide n by all integers from 2 to $\sqrt{n}$. If none divides evenly, n is prime. Numbers less than 2 are not prime.

```
import math
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0: return False
    return True

total = sum(i for i in range(2, 2000000) if is_prime(i))
print(total)
```

Q: Why do we only check up to $\sqrt{n}$ for primality?

A: If n has a factor greater than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$. So checking up to $\sqrt{n}$ is sufficient — this reduces time complexity from $O(n)$ to $O(\sqrt{n})$.

Q: What is the Sieve of Eratosthenes?

A: An efficient algorithm: create a boolean list of size $N+1$, initially True. For each prime p starting from 2, mark all multiples of p as False. Remaining True positions are primes. Much faster than testing each number individually.

Q: What is the Fibonacci sequence?

A: A sequence where each number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...

Q: How do you find sum of even-valued Fibonacci terms not exceeding 4 million?

```
a, b, total = 0, 1, 0
while a <= 4000000:
    if a % 2 == 0:
        total += a
    a, b = b, a + b
print('Sum of even Fibonacci terms:', total)
```

Topic 6 — Dictionary: Character Count & String Methods

Q: What is a dictionary in Python?

A: An unordered collection of **key-value pairs**. Keys must be unique and immutable. Values can be any type. Defined with `{}`. Lookup, insert, delete are $O(1)$ average.

```
person = {'name': 'Alice', 'age': 20, 'city': 'Jaipur'}
```

Q: How do you count character frequency in a string using a dictionary?

```
s = 'hello world'
freq = {}
```

```
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Q: What does `dict.get(key, default)` do?

A: Returns the value for key if it exists; otherwise returns the default value. This avoids `KeyError` when the key is missing.

Q: Explain the `split()` and `join()` string methods with examples.

A: **`split(sep)`**: splits a string into a list at each separator (default: whitespace). **`join(iterable)`**: joins elements of an iterable into a single string with the separator in between.

```
line = 'Alice 25 January'
parts = line.split()           # ['Alice', '25', 'January']
joined = '-'.join(parts)       # 'Alice-25-January'
print(joined)
```

Q: How would you store and display a person's birthday using a dictionary?

```
birthday = {'name': 'Ravi', 'day': 10, 'month': 'March', 'year': 2002}
print(f"{birthday['name']}'s birthday: {birthday['day']} {birthday['month']} {birthday['year']}")
```

Topic 7 — Character Frequency in Files & File Type Detection

Q: How do you count character frequency in a file?

```
freq = {}
with open('sample.txt', 'r') as f:
    for ch in f.read():
        freq[ch] = freq.get(ch, 0) + 1
for char, count in sorted(freq.items()):
    print(f"{char}: {count}")
```

Q: How can character frequency help identify file type?

A: **Python file**: high frequency of `#`, `:`, `def`, `import`, indentation spaces. **C file**: high frequency of `{`, `}`, `;`, `>`. **Text file**: mostly alphabets, spaces, periods — no special programming symbols.

Q: What are the file opening modes in Python?

Mode	Meaning
'r'	Read only (default). Error if file not found.
'w'	Write — creates new or overwrites existing.
'a'	Append — adds to end, creates if not exists.
'r+'	Read and write.
'rb'/'wb'	Binary read/write (images, PDFs, etc.)

Q: Why use `'with open()'` instead of just `open()`?

A: `'with'` is a context manager — it **automatically closes the file** when the block exits, even if an exception occurs. This prevents resource leaks and is the recommended practice.

Topic 8 — File Operations: Reverse Lines & Count Stats

Q: How do you print each line of a file in reverse order?

```
with open('file.txt', 'r') as f:
    lines = f.readlines()
```

```
for line in reversed(lines):
    print(line, end='')
```

Q: How do you count characters, words, and lines in a file?

```
with open('file.txt', 'r') as f:
    content = f.read()
    line_count = len(content.splitlines())
    word_count = len(content.split())
    char_count = len(content)
    print(f'Lines: {line_count}, Words: {word_count}, Chars: {char_count}')
```

Q: What is the difference between read(), readline(), and readlines()?

A: **read()**: returns the entire file content as a single string. **readline()**: reads one line at a time — useful for large files. **readlines()**: reads all lines into a list of strings.

Q: How do you reverse a string in Python?

A: Using slicing: `s[::-1]`. The step `-1` means go backwards. You can also use `".join(reversed(s))"`.

Topic 9 — Nearly Equal Strings & One-Line GCD / LCM

Q: What does 'nearly equal' mean for two strings?

A: Two strings `a` and `b` are nearly equal if `b` can be obtained from `a` by exactly **one mutation**: either inserting one character, deleting one character, or replacing one character (edit distance = 1).

Q: Write the nearly_equal function.

```
def nearly_equal(a, b):
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) == len(b):
        return sum(x != y for x, y in zip(a, b)) == 1
    if len(a) < len(b):
        a, b = b, a
    return any(a[:i] + a[i+1:] == b for i in range(len(a)))
```

Q: What is GCD and LCM?

A: **GCD** (Greatest Common Divisor): the largest integer that divides both numbers. **LCM** (Least Common Multiple): the smallest integer divisible by both. Relationship: $LCM(a,b) = a * b / GCD(a,b)$.

Q: Write one-line GCD and LCM functions using lambda.

```
import math
gcd = lambda a, b: math.gcd(a, b)
lcm = lambda a, b: a * b // math.gcd(a, b)

print(gcd(12, 18))    # 6
print(lcm(4, 6))      # 12
```

Q: What is a lambda function?

A: An anonymous, single-expression function. Syntax: **lambda args: expression**. Useful for short functions passed as arguments. Example: `sorted(lst, key=lambda x: x[1])`.

Topic 10 — Sorting Algorithms

Q: Explain Merge Sort. What is its time complexity?

A: Merge Sort is a **divide-and-conquer** algorithm. It recursively splits the array into halves until single elements, then merges sorted halves together. Time: $O(n \log n)$ for all cases. Space: $O(n)$. It is stable.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(L, R):
    result, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: result.append(L[i]); i += 1
        else: result.append(R[j]); j += 1
    return result + L[i:] + R[j:]
```

Q: Explain Selection Sort.

A: Finds the **minimum element** in the unsorted part and swaps it to the front. Repeats for remaining elements. Always $O(n^2)$ — not suitable for large data.

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Q: Explain Insertion Sort.

A: Picks each element and **inserts it into the correct position** in the already-sorted part — like sorting playing cards. Best case $O(n)$ (already sorted), worst case $O(n^2)$.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Q: Compare all three sorting algorithms.

Algorithm	Best Case	Average	Worst Case	Space	Stable
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

SECTION B — Additional Questions Teachers Commonly Ask

These go beyond the written syllabus but examiners ask them to test real understanding.

B1 — Python Basics

Q: What is Python? Name its key features.

A: Python is a high-level, interpreted, general-purpose language created by Guido van Rossum (1991). Key features: simple readable syntax, dynamically typed, garbage-collected memory, large standard library, supports OOP and functional programming.

Q: What does 'interpreted' mean?

A: Python code is executed line-by-line by the interpreter at runtime — it is not compiled to machine code first (unlike C/C++). This makes debugging easier but execution slightly slower.

Q: What is the difference between == and is?

A: == compares values (are the contents equal?). is compares identity (do both variables point to the same object in memory?). Two lists with same values: == is True but is is False.

```
a = [1,2,3]
b = [1,2,3]
print(a == b)  # True (same value)
print(a is b)  # False (different objects)
```

Q: What are mutable and immutable types? Give examples.

A: **Mutable** (can change): list, dict, set. **Immutable** (cannot change): int, float, str, tuple, bool. Immutable objects are safe as dictionary keys; mutable objects are not.

Q: What is the difference between append() and extend()?

A: **append(x)**: adds x as a single element at the end. **extend(iterable)**: adds every item from the iterable individually.

```
a = [1, 2]
a.append([3,4])  # [1, 2, [3, 4]] -- one element added
a = [1, 2]
a.extend([3,4])  # [1, 2, 3, 4] -- two elements added
```

B2 — Functions & Scope

Q: What is the difference between local and global variables?

A: **Local**: declared inside a function, accessible only within it. **Global**: declared outside all functions. To modify a global variable inside a function, use the **global** keyword.

```
x = 10
def change():
    global x
    x = 20
change()
print(x)  # 20
```

Q: What is recursion? What is a base case?

A: Recursion is when a function calls itself. The **base case** is the condition that stops the recursion — without it, you get infinite recursion and a RecursionError.

```
def factorial(n):
    if n == 0: return 1      # base case
    return n * factorial(n-1) # recursive call
```


Q: What is a list comprehension? Give an example.

A: A concise, Pythonic way to create a list. Syntax: [expression for item in iterable if condition].

```
evens = [x for x in range(20) if x % 2 == 0]
squares = [x**2 for x in range(10)]
```

Q: What are *args and **kwargs?

A: ***args**: allows a function to accept any number of positional arguments (stored as a tuple).

****kwargs**: allows any number of keyword arguments (stored as a dict).

```
def demo(*args, **kwargs):
    print(args)      # (1, 2, 3)
    print(kwargs)    # {'name': 'Alice'}
demo(1, 2, 3, name='Alice')
```

B3 — Exception Handling

Q: What is exception handling and why is it important?

A: Handling runtime errors gracefully so the program does not crash. Uses try-except blocks. Important for file I/O, user input, network calls — anything that can fail at runtime.

```
try:
    n = int(input('Enter number: '))
    print(10 / n)
except ValueError:
    print('Not a valid integer!')
except ZeroDivisionError:
    print('Cannot divide by zero!')
finally:
    print('This always runs.')
```

Q: What is the difference between a syntax error and a runtime error?

A: **Syntax error**: violation of Python grammar — caught BEFORE execution (e.g., missing colon, wrong indentation). **Runtime error**: occurs DURING execution — valid syntax but illegal operation (e.g., dividing by zero, accessing missing file).

Q: What does the 'finally' block do?

A: Code in finally ALWAYS executes, whether an exception occurred or not. Used to release resources — close files, close database connections, release locks.

B4 — Strings Deep Dive

Q: Are strings mutable in Python?

A: No — strings are **immutable**. You cannot change individual characters. Operations like replace() or upper() return a NEW string; the original is unchanged.

Q: Name 8 important string methods.

A: upper(), lower(), strip() (remove whitespace), replace(old,new), find(sub), count(sub), startswith(prefix), endswith(suffix), split(sep), join(list), isdigit(), isalpha(), title().

Q: How do you reverse a string? Can you do it without slicing?

```
s = 'Python'
reversed_s = s[::-1]                # 'nohtyP'
reversed_s2 = ''.join(reversed(s)) # 'nohtyP'
```

Q: What is string slicing? Explain s[1:5:2].

A: Slicing extracts a substring: `s[start:stop:step]`. `s[1:5:2]` starts at index 1, ends before index 5, takes every 2nd character.

B5 — Tricky Questions Examiners Love

Q: What is the output of: `print(0.1 + 0.2 == 0.3)`?

A: False! Due to floating point precision issues in binary representation. `0.1 + 0.2 = 0.30000000000000004` in Python. Use `round()` or `math.isclose()` for float comparisons.

Q: What will this print? `a=[1,2,3]; b=a; b.append(4); print(a)`

A: `[1, 2, 3, 4]` — because `b = a` does NOT copy the list; `b` points to the SAME list object. Modifying `b` also changes `a`. Use `b = a.copy()` or `b = list(a)` to get an independent copy.

Q: What is the difference between deep copy and shallow copy?

A: **Shallow copy**: copies the outer object but nested objects are still shared. **Deep copy**: copies everything recursively — fully independent. Use `copy.deepcopy()` for deep copy.

Q: What does `enumerate()` do?

A: Returns an iterator of (index, value) pairs. Avoids manually managing a counter variable.

```
for i, val in enumerate(['a', 'b', 'c'], start=1):  
    print(i, val)    # 1 a, 2 b, 3 c
```

Q: What is the difference between `pass`, `continue`, and `break`?

A: **pass**: does nothing — placeholder for empty blocks. **continue**: skips the rest of current iteration, goes to next. **break**: exits the loop entirely.

Q: What is the difference between a set and a list?

A: **Set**: unordered, no duplicates, $O(1)$ average lookup, uses `{}`. **List**: ordered, allows duplicates, $O(n)$ lookup, uses `[]`. Use sets when you need fast membership testing or unique elements.

Q: What is `zip()`? Give an example.

A: `zip()` pairs up elements from multiple iterables into tuples.

```
names = ['Alice', 'Bob', 'Carol']  
scores = [85, 90, 78]  
for name, score in zip(names, scores):  
    print(name, score)
```

BONUS — Quick Revision Cheat Sheet

Concept	Remember This
Data Types	int, float, str, bool, list, tuple, set, dict, NoneType
List vs Tuple	List = mutable [] Tuple = immutable ()
Dict keys	Must be immutable — str, int, tuple OK; list NOT OK
String	Immutable — all methods return NEW strings
Slicing	s[start:stop:step] — negative step reverses
File handling	Always use: with open(filename, mode) as f
Exception	try > except > else > finally
range()	range(start, stop, step) — stop is EXCLUDED
Sorting	Merge $O(n \log n)$ Selection $O(n^2)$ Insertion $O(n^2)$, best $O(n)$
GCD / LCM	$LCM(a,b) = a * b // GCD(a,b)$
Lambda	lambda x: x**2 (anonymous one-liner function)
*args/**kwargs	*args = positional tuple **kwargs = keyword dict
is vs ==	is checks identity (same object) == checks value equality
Shallow copy	b = a.copy() or b = list(a)
Deep copy	import copy; b = copy.deepcopy(a)
enumerate()	for i, v in enumerate(lst) — gives index + value
zip()	for a, b in zip(l1, l2) — pairs elements together

Best of Luck for Your Viva Exam!